

TABLE FORMAT NOTES

- Wide format
- Iceberg

G. Ronelli

Un buon table format deve essere **specifico**, deve essere **portabile** e **documentabile**. Deve supportare:

1. **ATOMIC CHANGES** o tale cancellata o nulla
2. **SCHEMA EVOLUTION** senza conseguenze non volute
3. **EFFICIENT ACCESS** come predizioni o proiezione in profondità.
4. **HIDDEN LAYOUT** non c'è bisogno di sapere la struttura dei dati (opzionale)
5. **LAYOUT EVOLUTION** possibilità di cambiare la struttura nel tempo (opzionale)

HIVE TABLES

L'idea è di organizzare i dati in **struttura a cartelle**, in cui le **PARTIZIONI** diventano **cartelle** con valori.

```
date=20180513/  
|- hour=18/  
|   |- ...  
|- hour=19/  
|   |- 000000_0  
|   |- ...  
|   |- 000031_0  
|- hour=20/  
|   |- ...  
|- ...
```

In questo modo è possibile usare queste colonne come **filter**.

Ad esempio, seando **date='20180513'** leggeremo **solo** quella cartella.

Ogni DB hive ha un **metastore** in cui tiene traccia di: 1. **partizioni** 2. **Informazioni sullo schema** 3. **Statistiche sulla tabella**

Esso permette di **filtrare** tramite valori di partizione, usare database esterni SQL, solo il file system traccia i file.

HIVE ACID LAYOUT

Fornisce **updates atomici e isolation**. Le **transazioni** sono salvate nel **metastore**. Usa lo stesso **partizion/Format Layout**.

DESIGN PROBLEM

- Lo stato della tabella è memorizzato in 2 posti: **Metastore** e **fileSystem**
- **Bucketing** è definito dall'implementazione Hive dell'Hash.
- L'unica operazione atomica del Non-Acid layout è l'aggiunta di partizioni
- Richiede lo spostamento atomico degli oggetti nel filesystem.
- Richiede la lista delle directory per pianificare i job
- I valori delle **partizioni** sono **salvati come stringhe**: - Richiede **Escape** - null salvati come **__HIVE_DEFAULT_PARTITION__**
- Le statistiche diventano obsolete e devono essere rigenerate manualmente
- Molte varianti di layout non documentati
- Le definizioni dei Bucket sono affidate a Hive e Hive
- L'utente deve conoscere il layout fisico
- Schema Evolution dipende dal formato del file
- I tipi cambiano anch'essi a seconda dei formati.

ICEBERG TABLES

L'idea è quella di tenere traccia di ogni file in una tabella nel corso del tempo:

- Uno snapshot è una lista completa di file in una tabella
- Ogni scrittura e commit creano un nuovo snapshot.

I benefici dello snapshot sono diversi:

- Isolamento dello snapshot senza lock: - Readers usano lo snapshot corrente - Writers creano nuovi snapshot
- Ogni cambio alla lista dei file è un'operazione atomica: - Aggiunta ed. tra le partizioni - Merge o riscrittura file

ICEBERG METADATA

Implementare il monitoraggio basato su snapshot: - Aggiungere table schema, partizioni e stringhe
- traccia i vecchi snapshot per garbage collection

- Ogni file dei metadata è immutabile

- Metadata avanzano sempre in avanti, lo storico è lineare

- Lo snapshot corrente può essere rolled back.

MANIFEST FILES

Un snapshot sono divisi in 1 o più manifest files: - Un manifest salva i file sulle varie partizioni
- Una tupla di ID di partizione sono salvati per ogni file
- Ri-usati per evitare alti volumi di scrittura.

Un manifest file contiene: - Info sui file: Location, formato, iceberg tracking data
- Valori per filtrare: valori partizioni, lower e upper bounds per colonna
- Metadati per ottimizzare: - FILE LEVEL: count e size - COLUMN LEVEL: count, null, size

COMMITTS

Prima di committare un writer deve: - Segnare la versione dei metadata attuale
- Creare un nuovo file di metadata e di manifest
- Cambiare la versione con la nuova atomicamente

Questo permette di avere uno storico lineare. Lo swap è implementato da un metastore custom ed un rename in HDFS.

Quando ci sono CONFLITTI si usa un writer ottimistico. Si assume che nessun altro stia scrivendo e in caso di conflitti si riprova con gli ultimi metadata.

BENEFICI

- Non ci sono operazioni onerose di File System
- Letture e scritture sono isolate ed atomiche.
- Scan più veloci: $O(1)$ manifest per la lettura, non $O(n)$ partition list
- Full Schema Evolution
- Supporto per i tipi
- Willen Partitioning
- Supporto per diversi file format

METADATA $\rightarrow$ Immutable! OR

Feature of Delta $\rightarrow$ Non è VACCOM

multiple table $\rightarrow$ No create

few reads $\rightarrow$ Merge on read

non è un table format $\rightarrow$ parquet

size iceberg $\rightarrow$ info in metadata

transaction log - json - version

metadata/index/schema - ?